INFORMATIK LK · WELT 6 · DS 37–40

Der Endgegner

Probeklausur, Fehlerjagd, Klausur, Abschluss



BOSS 6 ···· FEHLERJAGD ···· KLAUSUR ···· FINALE

Weltkarte: Der Endgegner



DS 37

Boss 6

Die Generalprobe

Probeklausur „Paketstation“: 35 BE in 90 min, Abi-Bedingungen

DS 38

Fehlerjagd

Korrektur & Top 10

Kernpunkte, Peer-Check, Team-Statistik, P1–P10 trainieren

DS 39–40

Klausur

„Ladepark“

Abitur-Format Block 1: 60 BE in 180 min – benotet

Finale

Abschluss

Scoreboard & Urkunden

Rückblick, Ränge, Abzeichen, Ausblick Abitur

Am Ende von Welt 6 …

- hast du eine Abiturprüfung im Kleinen erlebt – auf Zeit und auf Papier
- kennst du deine persönlichen Top-3-Punktverluste und ihre Gegenmittel
- hast du einen Abi-Fahrplan: Zeitplan, Vorgehen, eigener Abi-Check
- weißt du, wie du bis zum Abitur weitertrainierst

Algo Quest Lab in Welt 6

Fallenjagd: Level 6.1 · 6.2 (P1–P10)
Abi-Mix: Level 6.3 · 6.4
Online-Boss: Generalprobe online
Klausur: benotet – ohne Lab-Punkte

So läuft das Abitur ab: Block 1

60 BE
je Wahlaufgabe in Block 1

18 · 32 · 10
BE in den Anforderungsbereichen I · II · III

≈ 12 BE
davon Zusatzthema (z. B. Codierung, Automaten)

300 min
Prüfungszeit eA laut Aufgabenkopf



Aufgabe lesen
Einleitungstext zum Kontext, dann Teilaufgaben a), b), … mit Operator und [BE].



Material nutzen
Am Ende: Abbildungen, Klassenkarten, Struktogramme, Tabellen zum Ausfüllen.



Alles von Hand
Java-Code, Struktogramme nach §5, Trace- und Rekursionstabellen – ohne Compiler.



Hilfsmittel
die zugelassenen Abschnitte der Ergänzenden Hinweise (Schnittstellen, Notation).

Achtung Abi: „Laufzeitfehler sind durch geeignete Abfragen auszuschließen.“ – Dieser Satz steht bei jeder Implementierung.

Heute: Generalprobe „Paketstation“ – 35 BE · 90 min · gleiche Bedingungen wie im Abitur

Operatoren: Was wird von dir erwartet?

Operator	Erwartung (Operatorenliste)	So holst du die BE
nennen / angeben	ohne Erläuterung aufzählen	**kurz und genau das Gefragte**
beschreiben	in eigenen Worten mit Fachsprache wiedergeben	**Schritte in der richtigen Reihenfolge, Fachbegriffe**
erläutern	durch zusätzliche Informationen verständlich machen	**Wie? Warum? + Beispiel aus dem Material**
begründen / zeigen	auf Gesetzmäßigkeiten, kausale Zusammenhänge zurückführen	**„weil … deshalb …“ bis zur Regel**
dokumentieren / darstellen	Verlauf strukturiert wiedergeben	**nur Variablen, eine Zeile je markierter Stelle, Endwerte**
implementieren	Algorithmus codieren	**Signatur exakt, Leer-/Indexabfragen, Casts**
entwickeln / entwerfen	Algorithmus selbstständig erarbeiten	**Struktogramm nach §5 mit Signatur im Kopf**
analysieren / untersuchen	wichtige Eigenschaften herausarbeiten	**an einem Beispiel durchspielen, Ergebnis formulieren**
abschätzen	begründet Größenordnungen angeben	**Vergleiche zählen, Verdopplung, Rechenweg**
beurteilen	eigenes Urteil mit Fachwissen begründen	**Kriterien nennen → Urteil → Begründung**

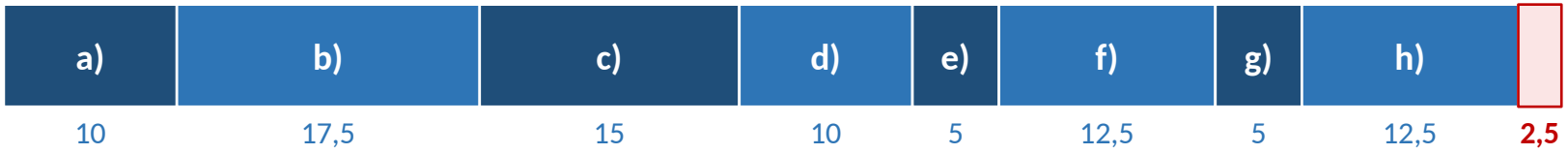
P10: Wer den Operator überliest, verschenkt BE – „erläutern“ ist mehr als nennen, „begründen“ mehr als beschreiben.

Zeitplanung: 2,5 Minuten je BE

BE	Zielzeit (min)
2	5
3	7,5
4	10
5	12,5
6	15
8	20
10	25
35	87,5
60	150

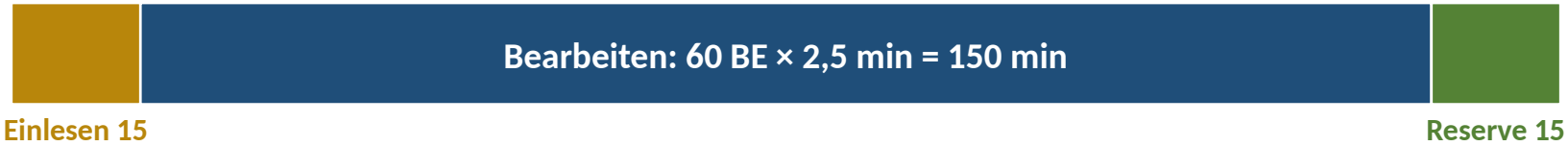
35 BE = Probeklausur · 60 BE = Klausur

Generalprobe heute: 90 min für 35 BE



Knapp! 35 BE × 2,5 min = 87,5 min – AFB-I-Teile (a, d) zügig holen, damit Zeit für Abi-Check und Reserve bleibt.

Klausur und Abitur-Format: 180 min für 60 BE



Zielzeiten an den Rand: Schreib beim Einlesen neben jede Teilaufgabe ihre Zielzeit und die Uhrzeit, bis zu der du fertig sein willst. Bist du 5 Minuten drüber: Lücke markieren, weiter, später zurück.

Prüfungsstrategie in vier Schritten

1



Überfliegen

3 min: Worum geht es? Welche Klassen, welche Abbildungen? Operatoren unterstreichen.

Probe: Paket, Fach, Paketstation – Abb. 1 bis 8

2



Material markieren

Klassenkarten, Signaturen, Beispielwerte und Tabellen zum Ausfüllen markieren.

Signatur abschreiben: sucheFach(sendungsnr: Zeichenkette): Ganzzahl

3



BE-Plan

Zielzeiten an den Rand (2,5 min je BE), Reihenfolge festlegen – Sicheres zuerst.

b) 7 BE → 17,5 min

4



Reserve

Zum Schluss Abi-Check je Methode, markierte Lücken schließen.

Leer? Index? Cast? equals? return?

Hängen geblieben? Lücke markieren, Zielzeit beachten, weiter zur nächsten Teilaufgabe – Teilpunkte gibt es für jeden Kernpunkt.

Boss 6: Die Generalprobe



90 Minuten · 35 BE
Teilaufgaben a) bis h), Operatoren wie im Abitur



Papier, ohne Gerät
eigene Klausurbögen; Tabellen in Abb. 4 und 7 im Material



Hilfsmittel
nur Ergänzende Hinweise (Auszug im Material)



Abi-Bedingungen
Einzeltische, Handy aus, keine Werkzeugkarten

Bewertung (Deckblatt)

	a)	b)	c)	d)	e)	f)	g)	h)	Σ
max. BE	4	7	6	4	2	5	2	5	**35**
selbst									
Peer									

Zusätzlich im Lab: Online-Boss „Generalprobe online“

Die Papier-Probe korrigierst du in DS 38 selbst und im Peer-Check – Abi-Training ohne Punkte.

Ziel heute: nicht perfekt sein – sondern herausfinden, wo deine Punkte liegen bleiben.

Korrektur nach Kernpunkten: Selbst → Peer → Team

1 · Selbstkorrektur

20 min

- Korrekturbogen aus dem Erwartungshorizont
- Spalte „selbst“ abhaken
- jeder fehlende Kernpunkt = Strich bei seiner P-Nummer

2 · Peer-Check

15 min

- Tausch im Ring A → B → C → A
- eigene Häkchen abdecken, Spalte „Peer“
- Abweichung? Einigung mit dem Kernpunkt-Text

3 · Team-Statistik

10 min

- Striche P1–P10 aller Mitglieder summieren
- Säulendiagramm, Top 3 markieren
- Foto = Vergleichsbasis für den Warm-up-Nachtest

So zählt ein Kernpunkt

- nur, wenn er vollständig und fachlich richtig dasteht – „gemeint“ zählt nicht
- Folgefehler: nur der Kernpunkt mit dem ersten Fehler geht verloren
- Lies wie ein Korrektor: Steht es da? Nicht: Hätte sie es gewusst?

Vergleich mit dem Lab

Du: Profil → „Deine Fallen“
Kurs: Fallen-Profil (Lehrkraft)
Peer-Check: alle Kernpunkte geprüft, Abweichungen begründet

Wie ein Korrektor liest: Teilaufgabe b)

Schülerlösung zu **`sucheFach(sendungsnr: Zeichenkette): Ganzzahl`**

```
1  public int sucheFach(String sendungsnr) {
2      for (int i = 0; i < faecher.getLength(); i++) {
3          Fach f = (Fach) faecher.getItem(i);
4          if (f.getInhalt().getNummer() == sendungsnr) {
5              return i;
6          }
7      }
8      return -1;
9  }
```

P1 · P5

P3

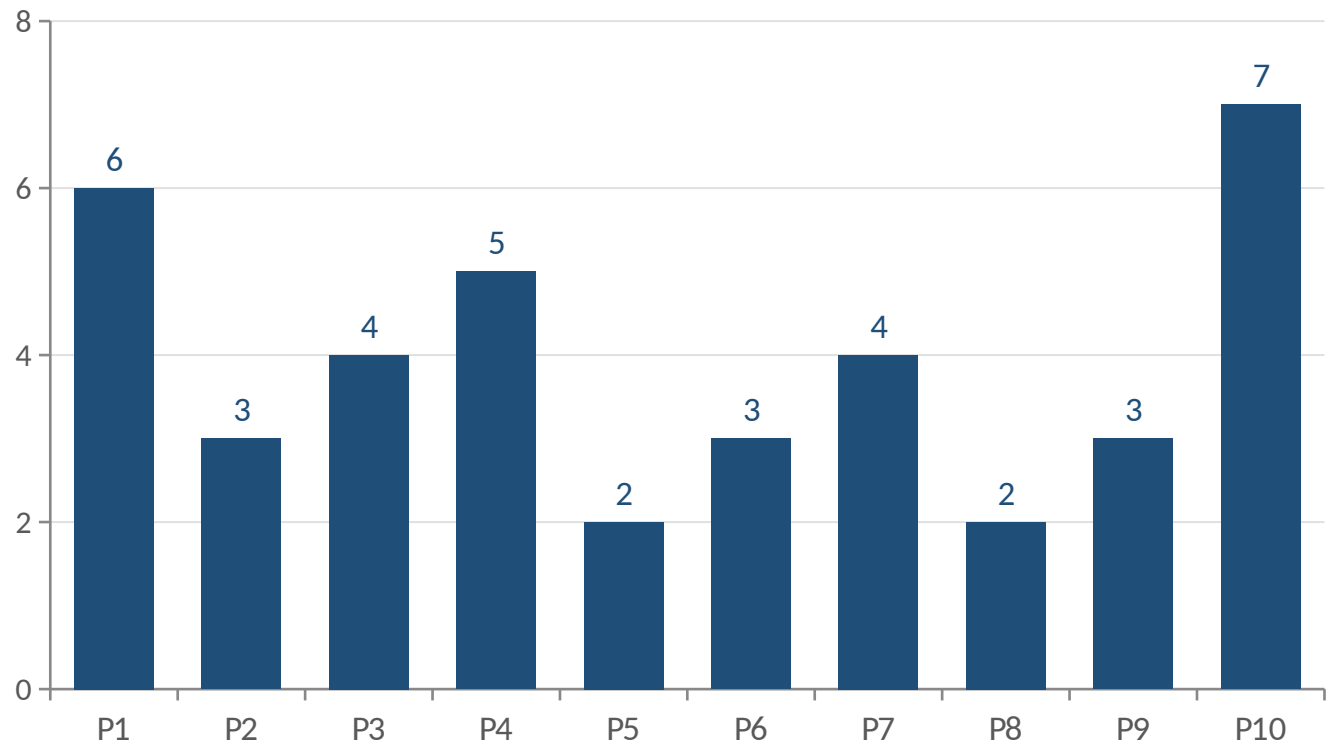
Papier-Compiler: `sucheFach("P-2208")` bricht ab – NullPointerException, weil Fach 102 frei ist (getInhalt() = null).

Und **==**? Die Online-IDE vergleicht Zeichenketten mit == inhaltlich – die Tests merken es nicht. Im Abitur kostet es trotzdem den Kernpunkt.

Kernpunkt b)	BE	erreicht	P
Methodenkopf passend zur Signatur	1	✓	P3
Durchlauf 0 bis getLength() − 1	1	✓	P6
getItem(i) mit Cast (Fach)	1	✓	P4
istFrei() vor getInhalt() geprüft	1	✗	**P1**
Vergleich mit equals	1	✗	**P5**
Fachnummer statt Index zurückgegeben	1	✗	**P3**
−1 nach vollständigem Durchlauf, jeder Pfad	1	✓	P3
Summe	**7**	**4**	

4 von 7 BE · 3 Striche: P1, P5, P3

Team-Statistik: Wo bleiben unsere Punkte?



Beispiel – eure echten Zahlen kommen aus dem AB, Teil 3

1 Striche sammeln
jedes Mitglied trägt seine Fehlerliste ein, ihr bildet die Teamsumme

2 Top 3 markieren
im Beispiel: P10, P1 und P4 – genau diese Fehlerjagd-Aufgaben zuerst

3 Kurs-Top-3
Teamsummen an der Tafel addieren: Was trainieren wir alle?

4 Lab-Vergleich
„Deine Fallen“ im Profil · Fallen-Profil des Kurses

Fair: Die Statistik zeigt Teams, keine Personen.

Die Top 10 Punktverluste

P1

Laufzeitfehler nicht ausgeschlossen

Leer-Abfrage vor top, pop, head, dequeue, getItem, getLeft; Index 0 … getLength() − 1

P2

Struktur zerstört statt unverändert

Stapel/Schlange danach nicht wiederhergestellt, obwohl gefordert

P3

Signatur verfehlt

Rückgabetyp, Parameter, Name, static/Sichtbarkeit; return fehlt auf einem Pfad

P4

Typen und Casts

(Integer) vergessen, Zeichen vs. Zeichenkette, ganzzahlige Division

P5

Zeichenketten mit == statt equals

== statt equals; nicht erlaubte String-Operationen

P6

Off-by-one

< statt <=, getLength() statt getLength() − 1, Startindex

P7

Rekursion

Basisfall fehlt/falsch, Ergebnis nicht verwendet, Tiefe falsch (Erstaufruf = 0)

P8

Trace-/ Dokumentationstabelle

nur Variablen, Zeile an jeder markierten Stelle, Endwerte

P9

Struktogramm nicht nach EH §5

←, „gib … zurück“, Schleifenarten, Verzweigung; Java im Struktogramm

P10

Operator verfehlt

„erläutern“ als bloße Nennung, „begründen“ ohne Grund, kein Materialbezug

Bug-Hunt: P1 und P2

P1 Laufzeitfehler nicht ausgeschlossen

Vorgabe: naechsteGroesse(): Zeichen – Größe des vordersten Auftrags, '-' bei leerer Schlange

✗ falsch

```
1  public char naechsteGroesse() {
2      Paket p = (Paket) auftraege.head();
3      return p.getGroesse();
4  }
```

✓ richtig

```
1  public char naechsteGroesse() {
2      if (auftraege.isEmpty()) {
3          return '-';
4      }
5      Paket p = (Paket) auftraege.head();
6      return p.getGroesse();
7  }
```

Gegenmaßnahme: Vor jedem head, dequeue, top, pop, getItem: Leer? Index? – als Frage an den Rand.

P2 Struktur zerstört statt unverändert

Vorgabe: anzahlRuecksendungen(): Ganzzahl – Anzahl der Pakete im Stapel ruecksendungen; der Stapel bleibt unverändert

✗ falsch

```
1  public int anzahlRuecksendungen() {
2      int n = 0;
3      Stack hilf = new Stack();
4      while (!ruecksendungen.isEmpty()) {
5          hilf.push(ruecksendungen.pop());
6          n++;
7      }
8      return n;
9  }
```

✓ richtig

```
// nach Zeile 7 einfügen:
    while (!hilf.isEmpty()) {
        ruecksendungen.push(hilf.pop());
    }
```

Gegenmaßnahme: „unverändert“ unterstreichen; Hilfsstruktur zurückfüllen, bevor return kommt.

Bug-Hunt: P3 und P4

P3 Signatur verfehlt

Vorgabe: hatFreiesFach(groesse: Zeichen): Wahrheitswert – wahr, wenn ein Fach der Größe groesse frei ist

× falsch

```
1  public boolean freiesFach(char groesse) {
2      for (int i = 0; i < faecher.getLength(); i++) {
3          Fach f = (Fach) faecher.getItem(i);
4          if (f.istFrei() && f.getGroesse() == groesse) {
5              return true;
6          }
7      }
8  }
```

✓ richtig

```
// Zeile 1 ersetzen durch:
public boolean hatFreiesFach(char groesse) {
// nach Zeile 7 einfügen:
    return false;
```

Gegenmaßnahme: Kopf zuerst 1 : 1 aus der Signatur übersetzen; jeden Pfad bis zum Ende verfolgen.

P4 Typen und Casts

Vorgabe: anteilFrei(): Kommazahl – Anteil der freien Fächer in Prozent; 0.0 bei einer Station ohne Fächer

× falsch

```
1  public double anteilFrei() {
2      int frei = 0;
3      for (int i = 0; i < faecher.getLength(); i++) {
4          Fach f = faecher.getItem(i);
5          if (f.istFrei()) { frei++; }
6      }
7      return frei / faecher.getLength() * 100;
8  }
```

✓ richtig

```
// Zeile 4 ersetzen durch:
        Fach f = (Fach) faecher.getItem(i);
// Zeile 7 ersetzen durch:
    if (faecher.getLength() == 0) {
        return 0.0;
    }
    return frei * 100.0 / faecher.getLength();
```

Gegenmaßnahme: Beim Entnehmen immer casten; Kommazahl? Mit 100.0 rechnen oder vorher casten.

Bug-Hunt: P5 und P6

P5 Zeichenketten mit == statt equals

Vorgabe: istExpress(p: Paket): Wahrheitswert – Sendungsnummern von Expresspaketen beginnen mit "PX-"

✗ falsch

```
1  public boolean istExpress(Paket p) {
2      return p.getNummer().substring(0, 3) == "PX-";
3  }
```

✓ richtig

```
1  public boolean istExpress(Paket p) {
2      String nr = p.getNummer();
3      return nr.length() >= 3
4             && nr.substring(0, 3).equals("PX-");
5  }
```

Gegenmaßnahme: Zeichenketten nur mit equals; nur Operationen aus den Ergänzenden Hinweisen.

P6 Off-by-one

Vorgabe: freiAb(start: Ganzzahl): Ganzzahl – Nummer des ersten freien Fachs ab Index start (start ≥ 0), sonst −1

✗ falsch

```
1  public int freiAb(int start) {
2      for (int i = start; i <= faecher.getLength(); i++) {
3          Fach f = (Fach) faecher.getItem(i);
4          if (f.istFrei()) {
5              return f.getNummer();
6          }
7      }
8      return -1;
9  }
```

✓ richtig

```
// Zeile 2 ersetzen durch:
    for (int i = start; i < faecher.getLength(); i++) {
```

Gegenmaßnahme: Grenzen mit dem ersten und letzten Index testen – und mit einer leeren/vollen Struktur.

Bug-Hunt: P7 und P8

P7 Rekursion

Vorgabe: anzahlZiffern(n: Ganzzahl): Ganzzahl – rekursiv, n ≥ 0, z. B. anzahlZiffern(4127) = 4

× falsch

```
1  public static int anzahlZiffern(int n) {
2      if (n < 10) {
3          return 1;
4      }
5      anzahlZiffern(n / 10);
6      return 1;
7  }
```

✓ richtig

```
// Zeilen 5–6 ersetzen durch:
    return 1 + anzahlZiffern(n / 10);
```

Erstaufruf = Tiefe 0

Rekursionstiefe	0	1	2	3
n	4127	412	41	4
Rückgabe	4	3	2	1

Gegenmaßnahme: Basisfall zuerst; Ergebnis des Aufrufs verwenden; Tabelle mit Tiefe 0 beginnen.

P8 Trace-/Dokumentationstabelle

Vorgabe: Tracetabelle nur mit Variablen – je eine Zeile an jeder Stelle „// → Tabellenzeile“ (Start, je Durchlauf, Ende).

```
int[] vol = {2, 3, 1, 3, 2};
int platz = 6;
int i = 0;   // → Tabellenzeile (Startwerte)
while (i < vol.length && vol[i] <= platz) {
    platz = platz - vol[i];
    i++;   // → Tabellenzeile
}   // → Tabellenzeile (Endwerte)
```

× falsch

Lauf	vol[i]	platz	i
Startzeile fehlt			
1	2	4	1
2	3	1	2
3	1	0	2
4	3	−3	4

✓ richtig

Lauf	vol[i]	platz	i
vor	–	6	0
1	2	4	1
2	3	1	2
3	1	0	3
Ende	3	0	3

Gegenmaßnahme: nur Variablen, Zeile an jeder markierten Stelle, Abbruch prüfen – genau die vorgegebenen Spalten.

Bug-Hunt: P9

P9 Struktogramm nicht nach EH §5

Vorgabe: belegtesVolumen(): Ganzzahl – Summe der Volumen aller eingelagerten Pakete. Beide rechnen richtig – nur eine ist ein Struktogramm.

× falsch

belegtesVolumen(): Ganzzahl

int summe = 0;

i = 0

while (i < faecher.getLength())

f = (Fach) faecher.getItem(i)

!f.istFrei()

wahr | falsch

summe += f.getInhalt().getVolumen()

i++

return summe;

nach EH §5

sprachunabhängig – kein Java!

✓ richtig

belegtesVolumen(): Ganzzahl

summe ← 0

für i von 0 bis faecher.getLength() − 1

f ← faecher.getItem(i)

nicht f.istFrei()

wahr | falsch

summe ← summe + f.getInhalt().getVolumen()

gib summe zurück

Java (falsch)	nach §5	Java (falsch)	nach §5
`int summe = 0;`	`summe ← 0`	`!f.istFrei()`	`nicht f.istFrei()`
`while … i++`	`für i von 0 bis … − 1`	`summe += …`	`summe ← summe + …`
`(Fach)-Cast`	`entfällt`	`return summe;`	`gib summe zurück`

Gegenmaßnahme: Nach dem Zeichnen jede Zeile prüfen: ←? „gib … zurück“? „für“/„solange“? „nicht“? Steht irgendwo ; == ++ && oder ein Cast?

Bug-Hunt: P10

P10 Operator verfehlt

Aufgabe: Begründen Sie, dass naechsteGroesse auch bei leerer Schlange keinen Laufzeitfehler verursacht. **[2 BE]**

```
1  public char naechsteGroesse() {
2      if (auftraege.isEmpty()) {
3          return '-';
4      }
5      Paket p = (Paket) auftraege.head();
6      return p.getGroesse();
7  }
```

Operator-Check

nennen: Stichwort genügt
beschreiben: Ablauf in Fachsprache
erläutern: Wie? Warum? + Beispiel aus dem Material
begründen: Ursache → Wirkung bis zur Regel

0 BE

„Weil es eine if-Abfrage gibt."

Kein Zusammenhang zu head() und zur leeren Schlange.

1 BE

„Die Methode prüft mit isEmpty(), ob die Schlange leer ist, und gibt dann '-' zurück."

Beschreibt nur – warum ist head() damit sicher?

2 BE

„Ein Laufzeitfehler entsteht nur, wenn head() auf eine leere Schlange angewendet wird. Ist auftraege leer, ist isEmpty() wahr und die Methode endet mit return '-', bevor head() erreicht wird."

Ursache → Wirkung, Bezug zum Code.

Gegenmaßnahme: Operator unterstreichen; bei „begründen" eine Weil-Kette bis zur Regel, immer mit Bezug zum Material oder Code.

Deine Checkliste: Mein Abi-Fahrplan

1 · Top-3-Fehler

Pro Fehler: So ist es mir passiert → meine Gegenmaßnahme → Training bis …

✗ *„besser aufpassen“*
✓ *„Vor jedem getItem schreibe ich die Indexgrenze an den Rand.“*

2 · Zeitplan

60 BE × 2,5 min = 150 min
180 − 150 = 30 min für Einlesen, BE-Plan und Reserve. Wo hast du in der Probe Zeit verloren?

3 · Vorgehen

Einlesen → BE-Plan → Bearbeiten → Abi-Check → Reserve
Reihenfolge festlegen – Sicheres zuerst, Hänger markieren.

4 · Mein Abi-Check

Leer? Index? Cast? equals? return auf jedem Pfad? Struktur unverändert? Basisfall? Struktogramm ohne Java?
+ deine eigenen Prüffragen

Papier-Compiler: Läuft deine Handschrift?

1. **Workspace öffnen**
   „W6 Papier-Compiler“ – Bibliothek „Niedersachsen“ aktiviert?

2. **Unverändert abtippen**
   deine Lösung zu b) Zeichen für Zeichen – auch mit Fehlern

3. **Testrunner starten**
   Ergebnis des ersten Laufs notieren, bevor du etwas änderst

4. **Fehlerlogbuch**
   jeden Fehler mit P-Nummer eintragen – dann korrigieren

Warum? Im Abitur gibt es keinen Compiler – nur deine Handschrift.

Teilaufgabe	Tests	Ziel
b) sucheFach (Pflicht)	aufgabeB_ (6)	**grün im 1. Lauf**
c) anzahlAuftraege (Kür)	aufgabeC_ (5)	**grün im 1. Lauf**
h) freieFaecher (Kür)	aufgabeH_ (6)	**grün im 1. Lauf**

c): Struktogramm Block für Block in Java übersetzen · h): deine Korrektur abtippen

Grün heißt nicht volle BE: == statt equals (P5) und eine Schleife statt Rekursion in h) finden die Tests nicht – das prüft der Korrekturbogen.

Ziel: Handschrift läuft beim ersten Abtippen

Letzte Runde vor der Klausur



Warm-up-Nachtest

Papier, ohne Gerät: Aufgaben zu den Kurs-Top-3. Das Team trägt seine Striche in die letzte Spalte der Strichliste ein.

Vergleich mit DS 38: Welche Fallen sind kleiner?



Fallen im Lab

Level 6.1 und 6.2 (Fallenjagd P1–P10), Level 6.3 und 6.4 (Abi-Mix) und der Online-Boss „Generalprobe online“ – automatisch geprüft.

Punkte gibt es dort automatisch



Gezielt trainieren

Dein Abi-Fahrplan: Top-3-Fehler mit Fehlerjagd-Aufgaben, Warm-up-Karten W6 im Warm-up-Trainer des Lab und den passenden Videos.

Endgegner-Mix: W6-01 bis W6-14

Achtung: Die Klausur wird benotet – Lab-Punkte zählen nicht für die Note. Punkte und Note bleiben getrennt, wie von Anfang an.

Klausur „Ladepark“: Das Briefing

180 min

einschließlich Einlesezeit

60 BE

AFB I · II · III = 18 · 32 · 10

Block 1

Abitur-Format, ohne Zusatzthema



Hilfsmittel
nur die zugelassenen Abschnitte der Ergänzenden Hinweise – sonst nichts



Handschriftlich
auf dem ausgegebenen Papier; Tabellen im Material ausfüllen und das Material mit abgeben



Java mit den Niedersachsen-Klassen
DynArray, Queue, Stack, BinTree sind nicht generisch – gelesene Inhalte casten



Laufzeitfehler ausschließen
leere Struktur, ungültiger Index, leerer Baum – durch explizite Abfragen

Mitbringen: dokumentenechte Stifte, Lineal fürs Struktogramm, Uhr ohne Netz. **Nachschreibtermin:** gleiches Format, andere Aufgabe.

Benotet – ohne Lab-Punkte. Dein Abi-Fahrplan und dein Abi-Check kommen mit – im Kopf.

Rückblick: Sieben Welten, sieben Bosse



WELT 0

Akademie

DS 1–7

Datentypen, Reihungen, eigene Methoden mit Signatur

BOSS 0
Die Bibliothek der Akademie



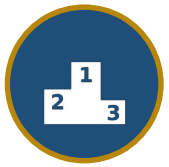
WELT 1

Heldenschmiede

DS 8–12

Klassen, Objekte, Vererbung, Klassendiagramm

BOSS 1
Die Gildenhalle



WELT 2

Rangliste

DS 13–17

Suchen, Sortieren, Effizienz durch Zählen

BOSS 2
Das Turnier der Bogenschützen



WELT 3

Labyrinth

DS 18–23

Rekursion, 2D-Reihungen, Teile und herrsche

BOSS 3
Die Kristallhöhle



WELT 4

Hafen

DS 24–30

Stapel, Schlange, dynamische Reihung

BOSS 4
Die Fähre von Nebelhafen



WELT 5

Wald

DS 31–36

Binärbaum, Suchbaum, unbekannte Bäume

BOSS 5
Die drei Baumhüter



WELT 6

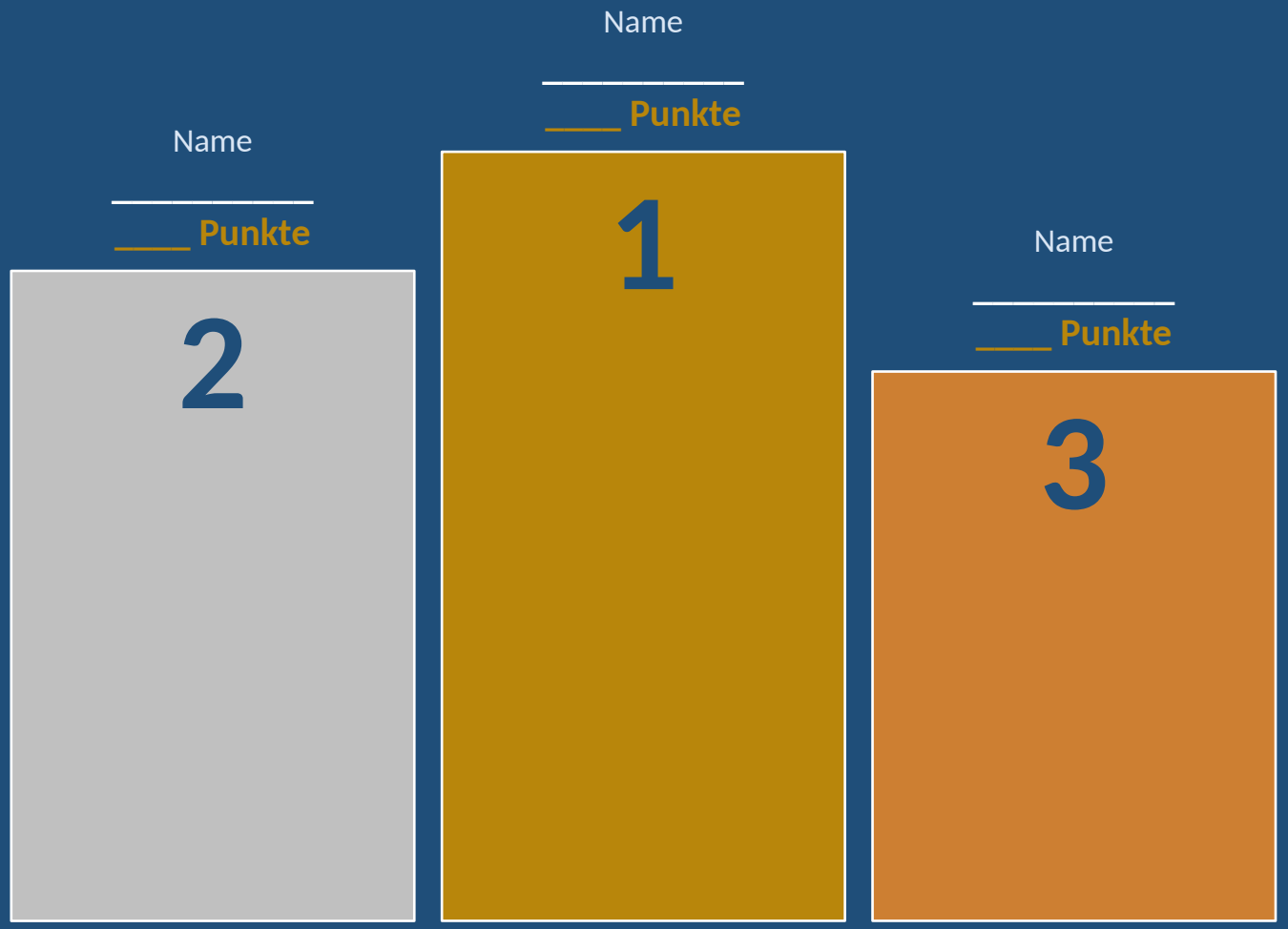
Endgegner

DS 37–40

Abitur-Format, Fehlerliste, Klausur

BOSS 6
Die Generalprobe

Finale: Das Scoreboard im Lab



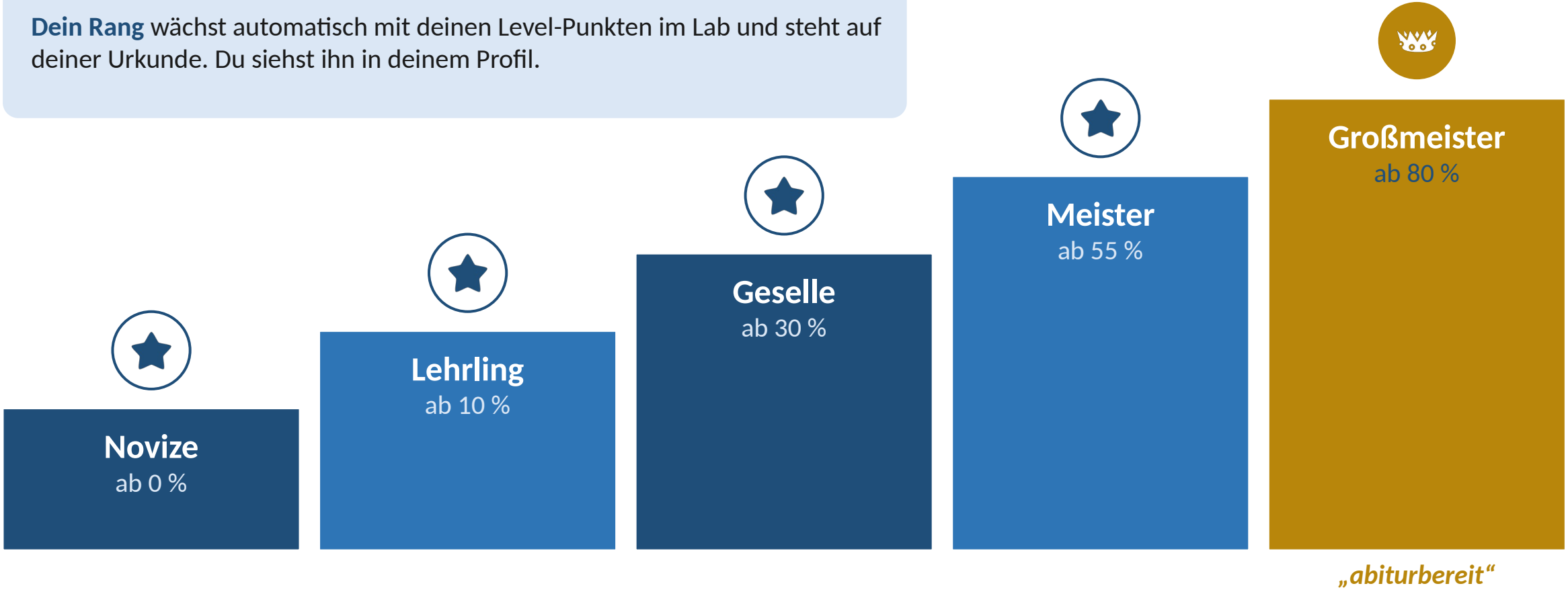
Scoreboard live
Lab → Scoreboard (gesamt oder je Welt) oder Beamer-Board: Punkte und Abzeichen, automatisch aus allen gelösten Aufgaben.

Größte Verbesserung im Kurs
Falle P ____ (DS 38 ↔ Nachtest)

Der ganze Kurs
_____ gelöste Lab-Aufgaben

Ränge: Wie weit bist du gekommen?

Dein Rang wächst automatisch mit deinen Level-Punkten im Lab und steht auf deiner Urkunde. Du siehst ihn in deinem Profil.



Schwellen: Anteil der Level-Punkte aller freigeschalteten Welten (Algo Quest Lab)

Abzeichen: Was habt ihr erreicht?



Erste Schritte

die erste Aufgabe gelöst

im Kurs: ____ ×



Zehnerserie

10 Aufgaben in Folge ohne Fehlversuch

im Kurs: ____ ×



Fleißig · Unermüdlich

50 bzw. 150 Aufgaben gelöst

im Kurs: ____ ×



Weltenbummler

in jeder Welt mindestens eine Aufgabe gelöst

im Kurs: ____ ×



„… gemeistert“

alle Level-Aufgaben einer Welt gelöst

im Kurs: ____ ×



„… besiegt“

Online-Boss einer Welt mit mindestens 50 % (Makellos: 100 %)

im Kurs: ____ ×



Fallensucher

zu jeder Falle P1–P10 drei Aufgaben ohne Fehlversuch

im Kurs: ____ ×



Warm-up-Meister

30 Warm-up-Karten in Fach 5 des Warm-up-Trainers

im Kurs: ____ ×

Urkunden aus dem Lab

Auf deiner Urkunde steht …

- dein Name im Lab und der Kurs
- dein Rang, deine Punkte und gelösten Aufgaben
- deine Abzeichen
- Datum und Unterschrift der Kursleitung
- optional: eine Zeile für deinen vollen Namen

Datenschutz: Das Lab speichert nur deinen selbst gewählten Namen (z. B. „Lena S.“) – den vollen Namen schreibst du selbst dazu.

Gedruckt im Lab: Lehrkraft → Kurs → „Urkunden“ – eine A4-Seite je Person.



Ausblick: Dein Weg ins Abitur

Algo Quest Lab

Warm-up-Trainer (Leitner) mit allen Karten aus Welt 0 bis 6, Trainingsplatz, Abi-Mix 6.3/6.4

2× pro Woche 10 min

YouTube-Playlist „Algo Quest“

Erklärvideos, Papier-Coding und Boss-Lösungen zum Nachschlagen

zu deinen Top-3-Fehlern

Abi-Aufgaben 2022–2026

echte Block-1-Aufgaben mit Erwartungshorizont

1 Aufgabe pro Monat, auf Zeit

Wiederholungsplan bis zum Abitur

Woche 1–4
Top-3-Fehler: Fehlerjagd-Aufgaben wiederholen, Warm-up W6

Monat 2–3
je Welt eine Abi-Teilaufgabe, Selbstkorrektur nach Kernpunkten

Monat 4–5
ganze Block-1-Aufgaben in 150 min + Papier-Compiler

vor dem Abi
Abi-Fahrplan aktualisieren, Abi-Check auswendig

Training mit echten Abi-Aufgaben 2022–2026

Welt	Typische Teilaufgaben	Zum Üben
Welt 1 · Heldenschmiede	Klasse und Konstruktoren erläutern, Klassendiagramm	`25H-B a · 22N-B a`
Welt 2 · Rangliste	gegebenes Sortierverfahren tracen und analysieren	`25H-B e/f · 24H-B d–g`
Welt 3 · Labyrinth	Rekursionstabelle, Teile und herrsche	`23N-C e · 24H-A d · 26H-B g`
Welt 4 · Hafen	Stapel/Schlange unverändert, dynamische Reihung	`25H-B b/c · 22N-A c/d · 26H-B e`
Welt 5 · Wald	Suchbaum, unbekannte Bäume (Heap, Huffman, B-Baum)	`22H-A a · 24N-A k/m · 25H-A · 26H-A`

Lesehilfe: `25H-B a` = Abitur 2025, Haupttermin, Aufgabe B, Teilaufgabe a) · N = Nachschreibtermin

So trainierst du: Zielzeit 2,5 min je BE stellen → auf Papier lösen → mit den Kernpunkten des Erwartungshorizonts selbst korrigieren → Striche P1–P10.

ALGO QUEST · ABSCHLUSS

Endgegner besiegt!

Das Abitur kann kommen – ihr seid bereit.



